

Stack

1. Overview

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle — the element inserted last is the first one to be removed. Think of it like a stack of plates: you place plates on top, and you remove the top plate first.

Stacks are one of the most fundamental data structures in computer science. They appear in virtually every placement exam, online assessment, and competitive programming contest. The concepts build from simple operations to advanced patterns like monotonic stacks, which unlock solutions to problems that would otherwise be $O(n^2)$ or worse.

Algorithms covered in this guide:

#	Algorithm / Problem	Core Pattern
1	Basic Stack Operations	LIFO, push, pop, top, empty
2	Balanced Parentheses	Matching pairs, validation
3	Next Greater Element	Monotonic stack (decreasing)
4	Previous Greater Element	Monotonic stack (decreasing, reverse)
5	Next Smaller Element	Monotonic stack (increasing)
6	Previous Smaller Element	Monotonic stack (increasing, reverse)
7	Monotonic Stack	Generic pattern for NGE/PGE/NSE/PSE
8	Min Stack	$O(1)$ getMin() with auxiliary stack
9	Evaluate Postfix/Prefix	Expression evaluation using stack
10	Largest Rectangle in Histogram	Monotonic stack (NSE + PSE)
11	Maximal Rectangle	2D $\rightarrow$ 1D reduction + histogram
12	Stock Span	NGE variant (consecutive smaller)
13	Asteroid Collision	Simulation with stack
14	Decode String	Nested decoding with two stacks
15	Remove Duplicate Letters	Lexicographically smallest with stack

2. Intuition

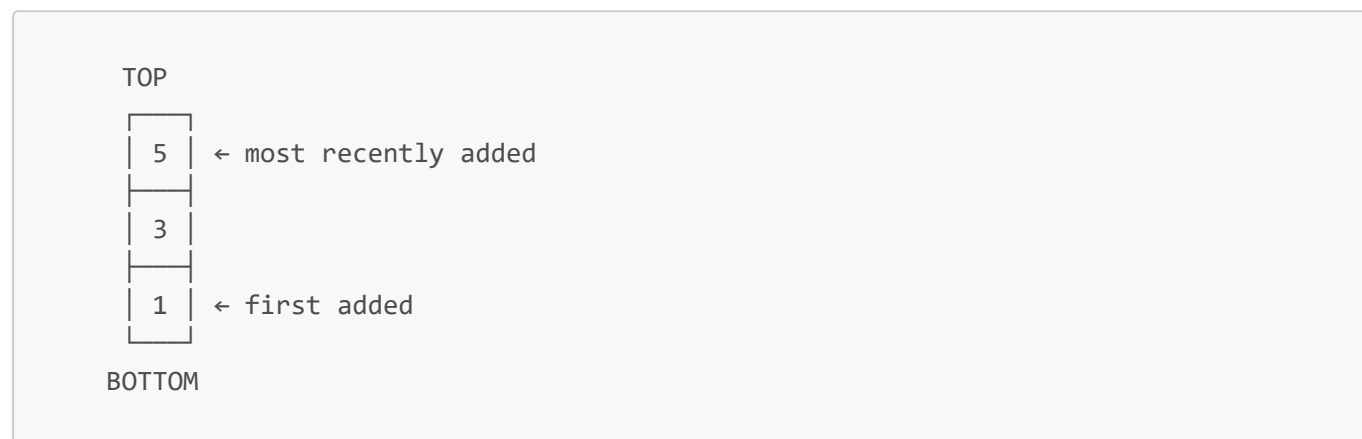
The LIFO Principle

A stack is like a **reversible action history**. Every push adds to the history, and every pop undoes the most recent action. This is why stacks power:

- **Undo/Redo** in editors
- **Call stack** in programming languages (function calls)
- **Back button** in browsers
- **Parsing** nested structures (HTML, JSON, mathematical expressions)

Core Mental Model

Imagine a **vertical tube** open only at the top.



- You can only look at the top element.
- You can only remove the top element.
- To add, you place on top.

Why This Matters for Algorithms

The stack is uniquely suited for problems where:

- **Order reversal** is needed (postfix evaluation)
- **Nested structures** must be validated (parentheses, JSON)
- **"Previous/Next"** relationships in arrays need to be computed efficiently (monotonic stack)
- **State needs to be saved** and revisited in reverse order (DFS, backtracking)

The Monotonic Stack Insight

The most powerful stack pattern is the **monotonic stack** — a stack that maintains elements in sorted order (strictly increasing or decreasing). This pattern replaces brute-force $O(n^2)$ "find next greater element" with $O(n)$ by exploiting the fact that when a new element breaks the monotonic property, we can resolve all pending elements.

The key insight: When iterating through an array, if we maintain a stack of elements in decreasing order, then whenever we see a larger element, it becomes the "next greater" for everything on the stack. This single idea unlocks 10+ problems.

3. When to Use It

General Stack Usage

- **Parsing / Validation:** Matching brackets, XML/HTML tags, JSON validation

- **Expression Evaluation:** Postfix, prefix, infix-to-postfix conversion
- **Reversal Problems:** Reverse a string, reverse a linked list in groups
- **Backtracking / DFS:** Storing path state, recursion simulation
- **Undo Operations:** Editor undo, browser history
- **Function Call Management:** Converting recursion to iteration

Monotonic Stack Usage

- **"Next Greater Element" type problems:** For each element, find the next element to the right that is larger/smaller
- **"Previous Greater Element" type problems:** For each element, find the previous element to the left that is larger/smaller
- **Histogram / Rectangle problems:** Largest rectangle, maximal rectangle, trapping rain water
- **Stock Span / Stock Price problems:** Consecutive smaller/larger prices
- **Sliding Window Maximum** (with deque): Related monotonic concept

Trigger Phrases in Problems

Phrase	Suggests
"Valid parentheses / brackets"	Stack for matching
"Next greater element"	Monotonic stack
"Nearest smaller to left/right"	Monotonic stack
"Largest rectangle in histogram"	Monotonic stack
"Stock span / consecutive days"	Monotonic stack
"Evaluate postfix / prefix expression"	Stack evaluation
"Decode string with nested brackets"	Stack with repetition
"Asteroid collision"	Stack simulation
"Remove duplicate letters / smallest subsequence"	Stack + frequency
"Min stack with $O(1)$ getMin"	Auxiliary stack
"Maximal rectangle in binary matrix"	Histogram per row

4. When Not to Use It

Simpler Alternatives

Situation	Better Choice
Need FIFO order	Queue / Deque
Need random access by index	Array / Vector
Need fast search / lookup	Hash Map / Set

Situation	Better Choice
Need min/max in sliding window	Deque (monotonic queue)
Need sorted order with min/max	Heap (Priority Queue)
Need range queries	Segment Tree / Fenwick Tree

Overkill Cases

- **Small fixed-size arrays:** A simple array with an index pointer is simpler and faster
- **Single pass, no backtracking needed:** Just use variables
- **Problem requires only min/max of entire set:** Min/max variables or heap suffice

Common Wrong Assumptions

- **"Stack always gives $O(1)$ operations"** → True for push/pop/top, but monotonic stack may pop multiple elements per push (amortized $O(1)$)
- **"Stack is always the answer for parentheses"** → Some problems need counting (no stack needed), e.g., "Minimum add to make parentheses valid" can be solved with a counter
- **"Monotonic stack is always increasing"** → Depends on problem: NGE uses decreasing stack, NSE uses increasing stack
- **"Stack is only for exact matching"** → Some problems need range checking or frequency tracking alongside stack

Edge Cases Where Stack Fails or Is Inefficient

- **Very deep recursion** → Stack overflow in real stack memory (use explicit stack or iterative approach)
- **Need $O(1)$ access to arbitrary elements** → Array is better
- **Multiple data structure queries** → Sometimes combining stack with hashmap or heap is needed
- **Large input with many duplicates** → Need careful handling in monotonic stack (strict vs non-strict)
- **Real-time / streaming constraints** → Stack is fine, but if window-based, deque may be better

5. Core Concepts

5.1 Basic Stack Operations

What it means: Push (insert), pop (remove), top (peek), empty (check if empty), size (number of elements).

Why it matters: These are the building blocks of every stack algorithm.

Example:

```
push(1) → [1]
push(2) → [1, 2]
push(3) → [1, 2, 3]
top() → 3
pop() → removes 3 → [1, 2]
empty() → false
```

5.2 Balanced Parentheses

What it means: Every opening bracket has a matching closing bracket in the correct order. `()` is valid, `)(` is not, `{[()]}` is valid.

Why it matters: This is the most common stack interview question. It tests understanding of LIFO for matching.

How it works: Push opening brackets onto stack. When a closing bracket is seen, check if stack top matches. If not, or stack is empty, invalid. At end, stack must be empty.

5.3 Next Greater Element (NGE)

What it means: For each element in an array, find the first element to its **right** that is **greater** than it. If none, return -1.

Why it matters: Introduces the monotonic stack pattern. The solution is $O(n)$ vs $O(n^2)$ brute force.

Core idea: Traverse from right to left (or left to right maintaining a decreasing stack). For each element, pop elements from stack that are $\leq$ current (they can never be NGE for anyone to the left). The top of stack is the NGE.

5.4 Previous Greater Element (PGE)

What it means: For each element, find the first element to its **left** that is **greater** than it.

Why it matters: Symmetric to NGE. Used in histogram problems and trapping rain water.

Core idea: Traverse from left to right. Maintain a decreasing stack. When a new element is larger than stack top, the stack top is the PGE... Actually, the standard approach: maintain a decreasing stack. For each element, while stack is not empty and stack top $\leq$ current, pop. The new top (if any) is the PGE.

5.5 Next Smaller Element (NSE)

What it means: For each element, find the first element to its **right** that is **smaller** than it.

Why it matters: Critical for "Largest Rectangle in Histogram" and similar problems.

Core idea: Maintain an **increasing** stack. When a new element is smaller than stack top, it is the NSE for the stack top.

5.6 Previous Smaller Element (PSE)

What it means: For each element, find the first element to its **left** that is **smaller** than it.

Why it matters: The other half of "Largest Rectangle in Histogram" — width = (NSE index - PSE index - 1).

Core idea: Traverse left to right with an increasing stack. For each element, pop while stack top $\geq$ current. The new top is the PSE.

5.7 Monotonic Stack

What it means: A stack that maintains elements in monotonic (strictly increasing or decreasing) order. When a new element violates the order, elements are popped until the order is restored.

Why it matters: This is a **design pattern**, not a specific algorithm. It reduces many $O(n^2)$ problems to $O(n)$.

Two variants:

Variant	Stack Order	Solves
Decreasing stack	Stack top is smallest	NGE, PGE
Increasing stack	Stack top is largest	NSE, PSE

Key insight: When you pop an element from the stack, you can determine something about it (e.g., its NGE is the current element being processed).

5.8 Min Stack

What it means: A stack that supports `getMin()` in $O(1)$ time alongside normal push/pop/top.

Why it matters: Classic interview problem testing auxiliary data structure design.

Approaches:

1. **Two stacks:** One main stack, one min stack that stores the current minimum at each level
2. **Single stack with pair:** `stack<pair<int, int>>` where second is min so far
3. **Single stack with encoding:** Store $2 * val - min$ to encode min in the value (saves space)

5.9 Evaluate Postfix / Prefix Expression

What it means: Given a mathematical expression in postfix (RPN) or prefix notation, evaluate it using a stack.

Why it matters: Tests understanding of stack-based computation. Postfix doesn't need parentheses or operator precedence.

Postfix evaluation: Scan left to right. Push operands. When operator is seen, pop two operands, apply operator, push result.

Prefix evaluation: Scan right to left. Same logic.

5.10 Largest Rectangle in Histogram

What it means: Given an array of bar heights, find the largest rectangle that can be formed in the histogram.

Why it matters: This is the **flagship monotonic stack problem**. It combines NSE and PSE concepts. Appears in top company interviews (Google, Amazon, Microsoft).

Core idea: For each bar, the largest rectangle that uses this bar has:

- Height = height of this bar
- Width = (NSE index - PSE index - 1)
- The maximum over all bars is the answer.

5.11 Maximal Rectangle

What it means: Given a binary matrix (0s and 1s), find the largest rectangle consisting entirely of 1s.

Why it matters: Extends histogram problem to 2D. Each row can be treated as a histogram where height = number of consecutive 1s above that cell.

Core idea: For each row, update heights: if cell is 1, height[i]++; else height[i] = 0. Run largest rectangle in histogram on each row's heights. Take the maximum.

5.12 Stock Span

What it means: For each day's stock price, find the number of consecutive days (including today) the price has been less than or equal to today's price.

Why it matters: Classic stack problem. It's essentially NGE/PGE in reverse.

Core idea: Maintain a stack of pairs (price, span). For each new price, pop all smaller prices and accumulate their spans. The span of the current price is the accumulated span + 1.

5.13 Asteroid Collision

What it means: Given an array of asteroids moving left (negative) or right (positive), simulate collisions. When two asteroids collide, the larger one survives; if equal, both are destroyed.

Why it matters: Tests stack simulation skills. The stack is used to keep track of asteroids moving right that may collide with incoming left-moving asteroids.

Core idea: Only push right-moving asteroids. When a left-moving asteroid is seen, repeatedly check the top of stack (right-moving asteroids) and resolve collisions.

5.14 Decode String

What it means: Given an encoded string like "3[a2[c]]", decode it to "accaccacc". The pattern is $k[\text{encoded_string}]$ where the encoded string is repeated k times.

Why it matters: Tests nested processing with stacks. Two stacks (or one stack with pairs) needed for numbers and strings.

Core idea: Use two stacks — one for counts, one for strings. When `[` is encountered, push current count and string. When `]` is encountered, pop and repeat.

5.15 Remove Duplicate Letters

What it means: Given a string, remove duplicate letters to get the smallest lexicographical subsequence containing all unique letters.

Why it matters: A more complex stack problem that combines frequency counting, visited tracking, and monotonic (lexicographic) order.

Core idea: Maintain a stack. For each character, if already in stack, skip. Otherwise, while stack is not empty, stack top > current character, and stack top appears later in the string, pop stack top. Push current character.

6. Step-by-Step Algorithm

6.1 Balanced Parentheses

Input: `s = "({[]})"`

1. Initialize empty stack
2. For each character `c` in `s`:
 - a. If `c` is opening bracket (`'('`, `'{'`, `'['`):
Push `c` onto stack
 - b. Else (`c` is closing bracket):
If stack is empty → return false
Pop top of stack
If top does not match `c` → return false
3. After loop: if stack is empty → return true, else false

6.2 Next Greater Element (NGE)

Input: `arr = [4, 5, 2, 25]`

1. Initialize empty stack, result array `res` of size `n` with `-1`
2. Traverse from left to right (or right to left):

Approach 1 (left to right, decreasing stack):

For `i = 0` to `n-1`:

While stack is NOT empty AND `arr[stack.top()] < arr[i]`:

`idx = stack.pop()`

`res[idx] = arr[i]`

Push `i` onto stack

Approach 2 (right to left):

For `i = n-1` down to `0`:

While stack is NOT empty AND `stack.top() <= arr[i]`:

`stack.pop()`

`res[i] = stack.top()` if stack not empty else `-1`

Push `arr[i]` onto stack

3. Return `res`

6.3 Previous Greater Element (PGE)

Input: arr = [4, 5, 2, 25]

1. Initialize empty stack, result array res of size n with -1
2. Traverse from left to right:
For i = 0 to n-1:
While stack is NOT empty AND stack.top() <= arr[i]:
stack.pop()
res[i] = stack.top() if stack not empty else -1
Push arr[i] onto stack
3. Return res

6.4 Next Smaller Element (NSE)

Input: arr = [4, 5, 2, 25]

1. Initialize empty stack, result array res of size n with -1
2. Traverse from left to right:
For i = 0 to n-1:
While stack is NOT empty AND arr[stack.top()] > arr[i]:
idx = stack.pop()
res[idx] = arr[i]
Push i onto stack
3. Return res

6.5 Previous Smaller Element (PSE)

Input: arr = [4, 5, 2, 25]

1. Initialize empty stack, result array res of size n with -1
2. Traverse from left to right:
For i = 0 to n-1:
While stack is NOT empty AND arr[stack.top()] >= arr[i]:
stack.pop()
res[i] = stack.top() if stack not empty else -1
Push i onto stack
3. Return res

6.6 Largest Rectangle in Histogram

Input: heights = [2, 1, 5, 6, 2, 3]

1. Compute NSE (next smaller element index) for each bar
2. Compute PSE (previous smaller element index) for each bar

```
3. For each bar i:
    width = NSE[i] - PSE[i] - 1
    area = heights[i] * width
    maxArea = max(maxArea, area)
4. Return maxArea
```

Alternative single-pass approach:

```
1. Push -1 to stack (sentinel)
2. For i = 0 to n:
    While heights[i] < heights[stack.top()]:
        h = heights[stack.pop()]
        w = (stack.empty() ? i : i - stack.top() - 1)
        maxArea = max(maxArea, h * w)
    Push i
3. Return maxArea
```

6.7 Evaluate Postfix Expression

Input: tokens = ["2", "1", "+", "3", "*"]

```
1. Initialize empty stack
2. For each token in tokens:
    If token is an operator (+, -, *, /):
        b = pop()
        a = pop()
        result = a operator b
        push(result)
    Else:
        push(int(token))
3. Return stack.top()
```

6.8 Stock Span

Input: prices = [100, 80, 60, 70, 60, 75, 85]

```
1. Initialize empty stack, result array span of size n
2. For i = 0 to n-1:
    While stack is NOT empty AND prices[stack.top()] <= prices[i]:
        stack.pop()
    span[i] = (stack.empty() ? i + 1 : i - stack.top())
    Push i onto stack
3. Return span
```

6.9 Asteroid Collision

Input: asteroids = [5, 10, -5]

1. Initialize empty stack
2. For each asteroid in asteroids:
 - If asteroid > 0 (moving right):
 - Push asteroid
 - Else (moving left, negative value):
 - While stack is NOT empty AND stack.top() > 0 AND stack.top() < -asteroid:
 - stack.pop() // smaller right-moving asteroid destroyed
 - If stack is empty OR stack.top() < 0:
 - Push asteroid // no collision, or all right-movers destroyed
 - Elif stack.top() == -asteroid:
 - stack.pop() // both destroyed
3. Return stack (as array)

6.10 Decode String

Input: s = "3[a2[c]]"

1. Initialize two stacks: countStack, stringStack
2. Initialize currentString = "", currentCount = 0
3. For each character c in s:
 - If c is digit:
 - currentCount = currentCount * 10 + (c - '0')
 - Elif c == '[':
 - Push currentCount to countStack
 - Push currentString to stringStack
 - Reset currentString = "", currentCount = 0
 - Elif c == ']':
 - count = countStack.pop()
 - prevString = stringStack.pop()
 - currentString = prevString + (currentString repeated count times)
 - Else (letter):
 - currentString += c
4. Return currentString

6.11 Remove Duplicate Letters

Input: s = "cbacdcbc"

1. Count frequency of each character in s
2. Initialize empty stack, visited boolean array for 26 chars
3. For each character c in s:
 - Decrement frequency of c
 - If visited[c] is true → continue
 - While stack is NOT empty AND stack.top() > c AND freq[stack.top()] > 0:
 - visited[stack.top()] = false
 - stack.pop()

```
    Push c onto stack
    visited[c] = true
4. Build string from stack (bottom to top)
5. Return result
```

7. Dry Run

7.1 Balanced Parentheses

Input: `s = "({[]})"`

Step	Char	Stack Before	Action	Stack After
1	(	[]	Push	[(
2	{	[(	Push	[(, {
3	[	[(, {	Push	[(, {, [
4	]	[(, {, [	Pop, matches]	[(, {
5	}	[(, {	Pop, matches }	[(
6	)	[(	Pop, matches)	[]

Result: Stack is empty → **Valid** ✓

7.2 Next Greater Element (Left to Right)

Input: `arr = [4, 5, 2, 25]`

i	arr[i]	Stack (indices)	Action	Result
0	4	[]	Push 0	[0]
1	5	[0]	arr[0] < 5 → pop 0, res[0]=5	[]
1	5	[]	Push 1	[1]
2	2	[1]	arr[1] > 2 → push 2	[1, 2]
3	25	[1, 2]	arr[2] < 25 → pop 2, res[2]=25	[1]
3	25	[1]	arr[1] < 25 → pop 1, res[1]=25	[]
3	25	[]	Push 3	[3]

Result: `[5, 25, 25, -1]`

7.3 Largest Rectangle in Histogram

Input: `heights = [2, 1, 5, 6, 2, 3]`

Step 1: Compute NSE indices

Index	Height	NSE Index	Why
0	2	1	$\text{heights}[1] = 1 < 2$
1	1	6 (sentinel)	No smaller to right
2	5	4	$\text{heights}[4] = 2 < 5$
3	6	4	$\text{heights}[4] = 2 < 6$
4	2	6	No smaller to right
5	3	6	No smaller to right

Step 2: Compute PSE indices

Index	Height	PSE Index	Why
0	2	-1 (sentinel)	No smaller to left
1	1	-1	No smaller to left
2	5	1	$\text{heights}[1] = 1 < 5$
3	6	2	$\text{heights}[2] = 5 < 6$
4	2	1	$\text{heights}[1] = 1 < 2$
5	3	4	$\text{heights}[4] = 2 < 3$

Step 3: Compute area for each bar

Index	Height	PSE	NSE	Width = NSE - PSE - 1	Area
0	2	-1	1	$1 - (-1) - 1 = 1$	2
1	1	-1	6	$6 - (-1) - 1 = 6$	6
2	5	1	4	$4 - 1 - 1 = 2$	10
3	6	2	4	$4 - 2 - 1 = 1$	6
4	2	1	6	$6 - 1 - 1 = 4$	8
5	3	4	6	$6 - 4 - 1 = 1$	3

Max Area = 10 ✅ (the rectangle formed by $\text{heights}[2]=5$ and $\text{heights}[3]=6$, width 2)

7.4 Evaluate Postfix

Input: `tokens = ["2", "1", "+", "3", "*"]`

Step	Token	Stack Before	Action	Stack After
1	"2"	<code>[]</code>	Push 2	<code>[2]</code>
2	"1"	<code>[2]</code>	Push 1	<code>[2, 1]</code>

Step	Token	Stack Before	Action	Stack After
3	"+"	[2, 1]	Pop 1, Pop 2 → 2+1=3, Push 3	[3]
4	"3"	[3]	Push 3	[3, 3]
5	"*"	[3, 3]	Pop 3, Pop 3 → 3*3=9, Push 9	[9]

Result: 9 

7.5 Stock Span

Input: prices = [100, 80, 60, 70, 60, 75, 85]

i	Price	Stack Before	Action	Stack After	Span
0	100	[]	Push 0	[0]	1
1	80	[0]	100 > 80, push 1	[0, 1]	1
2	60	[0, 1]	80 > 60, push 2	[0, 1, 2]	1
3	70	[0, 1, 2]	60 ≤ 70, pop 2	[0, 1]	
3	70	[0, 1]	80 > 70, push 3	[0, 1, 3]	3-1=2
4	60	[0, 1, 3]	70 > 60, push 4	[0, 1, 3, 4]	1
5	75	[0, 1, 3, 4]	60 ≤ 75, pop 4	[0, 1, 3]	
5	75	[0, 1, 3]	70 ≤ 75, pop 3	[0, 1]	
5	75	[0, 1]	80 > 75, push 5	[0, 1, 5]	5-1=4
6	85	[0, 1, 5]	75 ≤ 85, pop 5	[0, 1]	
6	85	[0, 1]	80 ≤ 85, pop 1	[0]	
6	85	[0]	100 > 85, push 6	[0, 6]	6-0=6

Result: [1, 1, 1, 2, 1, 4, 6]

7.6 Asteroid Collision

Input: asteroids = [5, 10, -5]

Step	Asteroid	Stack Before	Action	Stack After
1	5	[]	Right-moving, push	[5]
2	10	[5]	Right-moving, push	[5, 10]
3	-5	[5, 10]	10 > 0, 10 > 5, pop 10	[5]
3	-5	[5]	5 > 0, 5 > 5? No (5 == 5),	[5]
3	-5	[5]	5 > 0, 5 > 5? No, so push -5? No, 5 >	-5

Wait, let me redo: -5 (moving left). Stack top is 10 (moving right). $10 > |-5| = 5$, so -5 is destroyed. Stack stays [5, 10]. Then we move to next asteroid. But wait, we need to check again with the new top (5). $5 > |-5| = 5$? No, they're equal. Actually, 5 is not greater than 5, so -5 is destroyed. Wait no - the condition is: if the right-moving asteroid is **larger** than the left-moving one, the left one is destroyed. If they're equal, both are destroyed. If the right-moving is smaller, it's destroyed.

Actually, let me redo properly:

Step	Asteroid	Stack Before	Action	Stack After
1	5	[]	Right-moving ($5 > 0$), push	[5]
2	10	[5]	Right-moving ($10 > 0$), push	[5, 10]
3	-5	[5, 10]	Left-moving, top=10, $10 > 0$, $10 > 5$, pop 10	[5]
3	-5	[5]	Left-moving, top=5, $5 > 0$, $5 > 5$? No (equal). Both destroyed? No, 5 is not greater than 5. So -5 is destroyed.	[5] (unchanged)

Hmm, let me reconsider. The standard asteroid collision problem:

- If $top > 0$ (right-moving) and $asteroid < 0$ (left-moving), collision happens
- If $|top| < |asteroid|$, top is destroyed (pop)
- If $|top| == |asteroid|$, both destroyed (pop and skip)
- If $|top| > |asteroid|$, asteroid is destroyed (skip)

For step 3: asteroid = -5, stack top = 10. $10 > 0$, $-5 < 0$. $10 > 5$, so 10 is destroyed (pop). Stack becomes [5]. Then check again: top = 5, $5 > 0$, $-5 < 0$. $5 > 5$? No, $5 == 5$. Both destroyed. Pop 5 and discard -5. Stack becomes [].

Result: []

7.7 Decode String

Input: `s = "3[a2[c]]"`

Step	Char	countStack	stringStack	currCount	currString
1	3	[]	[]	3	""
2	[	[3]	[""]	0	""
3	a	[3]	[""]	0	"a"
4	2	[3]	[""]	2	"a"
5	[	[3, 2]	["", "a"]	0	""
6	c	[3, 2]	["", "a"]	0	"c"
7	]	[3]	[""]	0	"a" + "c"*2 = "acc"

Step	Char	countStack	stringStack	currCount	currString
8	]	[]	[]	0	"" + "acc"*3 = "accaccacc"

Result: "accaccacc" 

7.8 Remove Duplicate Letters

Input: $s = \text{"cbacdcbc"}$ (freq: c:4, b:2, a:1, d:1)

Step	Char	Stack	visited	Action
1	c	[]	{}	freq[c]=3, push c
2	b	[c]	{c}	freq[b]=1, $c > b$, freq[c]=3>0, pop c, visited[c]=false
2	b	[]	{}	push b
3	a	[b]	{b}	freq[a]=0, $b > a$, freq[b]=1>0, pop b, visited[b]=false
3	a	[]	{}	push a
4	c	[a]	{a}	freq[c]=2, $a < c$, push c
5	d	[a, c]	{a, c}	freq[d]=0, $c < d$, push d
6	c	[a, c, d]	{a, c, d}	freq[c]=1, visited[c]=true, skip
7	b	[a, c, d]	{a, c, d}	freq[b]=0, $d > b$, freq[d]=0 (no more), push b?
7	b	[a, d]	{a, d}	$d > b$, freq[d]=0, don't pop. $a < b$, push b
8	c	[a, d, b]	{a, d, b}	freq[c]=0, $b < c$, push c? But also check: $d < c$, push?

Wait, let me redo this more carefully. The frequencies: c:4, b:2, a:1, d:1.

$s = \text{"c b a c d c b c"} \quad 0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7$

freq = {c:4, b:2, a:1, d:1}

Step	i	Char	freq left	visited	Stack	Action
1	0	c	c:3	{}	[]	push c
2	1	b	b:1	{c}	[c]	$c > b$, freq[c]=3>0, pop c, visited[c]=false
2	1	b	b:1	{}	[]	push b
3	2	a	a:0	{b}	[b]	$b > a$, freq[b]=1>0, pop b, visited[b]=false
3	2	a	a:0	{}	[]	push a
4	3	c	c:2	{a}	[a]	$a < c$, push c
5	4	d	d:0	{a, c}	[a, c]	$c < d$, push d

Step	i	Char	freq left	visited	Stack	Action
6	5	c	c:1	{a, c, d}	[a, c, d]	visited[c]=true, skip
7	6	b	b:0	{a, c, d}	[a, c, d]	d>b, freq[d]=0, don't pop. c>b, freq[c]=1>0, pop c, visited[c]=false
7	6	b	b:0	{a, d}	[a, d]	d>b, freq[d]=0, don't pop. a<b, push b
8	7	c	c:0	{a, d, b}	[a, d, b]	b<c, push c

Result: "adbcb" 

8. C++ Implementation

8.1 Basic Stack (STL)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;

    st.push(10);    // [10]
    st.push(20);    // [10, 20]
    st.push(30);    // [10, 20, 30]

    cout << "Top: " << st.top() << "\n";    // 30
    cout << "Size: " << st.size() << "\n";    // 3

    st.pop();        // [10, 20]
    cout << "After pop, top: " << st.top() << "\n";    // 20

    cout << "Is empty: " << (st.empty() ? "Yes" : "No") << "\n";    // No

    // Iterating (destructive)
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    cout << "\n";

    return 0;
}
```

8.2 Balanced Parentheses

```

#include <bits/stdc++.h>
using namespace std;

bool isValid(string s) {
    stack<char> st;

    for (char c : s) {
        if (c == '(' || c == '{' || c == '[') {
            st.push(c);
        } else {
            if (st.empty()) return false;

            char top = st.top();
            st.pop();

            if ((c == ')' && top != '(') ||
                (c == '}' && top != '{') ||
                (c == ']' && top != '[')) {
                return false;
            }
        }
    }

    return st.empty();
}

int main() {
    vector<string> tests = {"()", "()[]{}", "()", "([])", "{[]}"};
    for (string s : tests) {
        cout << s << " -> " << (isValid(s) ? "Valid" : "Invalid") << "\n";
    }
    return 0;
}

```

8.3 Next Greater Element (NGE)

```

#include <bits/stdc++.h>
using namespace std;

// Left to right approach (decreasing stack of indices)
vector<int> nextGreaterElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] < arr[i]) {
            res[st.top()] = arr[i];
            st.pop();
        }
    }
}

```

```

        st.push(i);
    }

    return res;
}

int main() {
    vector<int> arr = {4, 5, 2, 25};
    vector<int> res = nextGreaterElement(arr);

    for (int i = 0; i < arr.size(); i++) {
        cout << arr[i] << " -> " << res[i] << "\n";
    }
    return 0;
}

```

8.4 Previous Greater Element (PGE)

```

#include <bits/stdc++.h>
using namespace std;

vector<int> previousGreaterElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st; // stores values (or indices)

    for (int i = 0; i < n; i++) {
        while (!st.empty() && st.top() <= arr[i]) {
            st.pop();
        }
        res[i] = st.empty() ? -1 : st.top();
        st.push(arr[i]);
    }

    return res;
}

```

8.5 Next Smaller Element (NSE)

```

#include <bits/stdc++.h>
using namespace std;

vector<int> nextSmallerElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] > arr[i]) {

```

```

        res[st.top()] = arr[i];
        st.pop();
    }
    st.push(i);
}

return res;
}

```

8.6 Previous Smaller Element (PSE)

```

#include <bits/stdc++.h>
using namespace std;

vector<int> previousSmallerElement(const vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] >= arr[i]) {
            st.pop();
        }
        res[i] = st.empty() ? -1 : arr[st.top()];
        st.push(i);
    }

    return res;
}

```

8.7 Monotonic Stack Template

```

#include <bits/stdc++.h>
using namespace std;

// Generic monotonic stack template
// isIncreasing: true for increasing stack (NSE/PSE), false for decreasing
// (NGE/PGE)
// isLeft: true for left-to-right (PSE/PGE), false for right-to-left (NSE/NGE)
// strict: true for strict comparison, false for non-strict
template<bool isIncreasing, bool isLeft, bool strict = true>
vector<int> monotonicStack(const vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    stack<int> st;

    auto cmp = [](int a, int b) {
        if constexpr (isIncreasing) return strict ? a >= b : a > b;
        else return strict ? a <= b : a < b;
    };

```

```

};

int start = isLeft ? 0 : n - 1;
int end = isLeft ? n : -1;
int step = isLeft ? 1 : -1;

for (int i = start; i != end; i += step) {
    while (!st.empty() && cmp(arr[st.top()], arr[i])) {
        // For right-to-left, the current element is the answer for popped
elements
        if constexpr (!isLeft) {
            // res[st.top()] is set when we resolve
            // Actually NGE right-to-left works differently
        }
        st.pop();
    }

    // For left-to-right, the stack top is the answer for current element
    if constexpr (isLeft) {
        res[i] = st.empty() ? -1 : arr[st.top()];
    } else {
        res[i] = st.empty() ? -1 : arr[st.top()];
    }

    st.push(i);
}

return res;
}

```

8.8 Min Stack

```

#include <bits/stdc++.h>
using namespace std;

class MinStack {
private:
    stack<int> st;        // main stack
    stack<int> minSt;    // auxiliary stack for minimums

public:
    void push(int val) {
        st.push(val);
        if (minSt.empty() || val <= minSt.top()) {
            minSt.push(val);
        }
    }

    void pop() {
        if (st.top() == minSt.top()) {
            minSt.pop();
        }
    }
}

```

```

    }
    st.pop();
}

int top() {
    return st.top();
}

int getMin() {
    return minSt.top();
}
};

int main() {
    MinStack ms;
    ms.push(3);
    ms.push(5);
    cout << "Min: " << ms.getMin() << "\n"; // 3
    ms.push(2);
    ms.push(1);
    cout << "Min: " << ms.getMin() << "\n"; // 1
    ms.pop();
    cout << "Min: " << ms.getMin() << "\n"; // 2
    ms.pop();
    cout << "Top: " << ms.top() << "\n"; // 5
    return 0;
}

```

8.9 Evaluate Postfix Expression

```

#include <bits/stdc++.h>
using namespace std;

int evaluatePostfix(vector<string>& tokens) {
    stack<int> st;

    for (string& token : tokens) {
        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();

            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
            else if (token == "/") st.push(a / b); // integer division
        } else {
            st.push(stoi(token));
        }
    }

    return st.top();
}

```

```

}

// Evaluate Prefix (scan right to left)
int evaluatePrefix(vector<string>& tokens) {
    stack<int> st;

    for (int i = tokens.size() - 1; i >= 0; i--) {
        string& token = tokens[i];
        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int a = st.top(); st.pop();
            int b = st.top(); st.pop();

            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
            else if (token == "/") st.push(a / b);
        } else {
            st.push(stoi(token));
        }
    }

    return st.top();
}

int main() {
    vector<string> postfix = {"2", "1", "+", "3", "*"};
    cout << "Postfix result: " << evaluatePostfix(postfix) << "\n"; // 9

    vector<string> prefix = {"*", "+", "2", "1", "3"};
    // In prefix, this is: * + 2 1 3 = (2+1)*3 = 9
    // But our function expects operands in order... let me check
    // Actually prefix: * + 2 1 3 → scan right to left: push 3, push 1, push 2
    // Then +: pop 2, pop 1, push 3. Then *: pop 3, pop 3, push 9. Result: 9 ✓
    cout << "Prefix result: " << evaluatePrefix(prefix) << "\n"; // 9
    return 0;
}

```

8.10 Largest Rectangle in Histogram

```

#include <bits/stdc++.h>
using namespace std;

// Single-pass approach (most efficient)
int largestRectangleArea(vector<int>& heights) {
    int n = heights.size();
    stack<int> st;
    int maxArea = 0;

    for (int i = 0; i <= n; i++) {
        int currHeight = (i == n) ? 0 : heights[i];

```

```

        while (!st.empty() && heights[st.top()] > currHeight) {
            int h = heights[st.top()];
            st.pop();

            int left = st.empty() ? -1 : st.top();
            int width = i - left - 1;
            maxArea = max(maxArea, h * width);
        }

        st.push(i);
    }

    return maxArea;
}

// Two-pass approach (using NSE and PSE)
int largestRectangleAreaTwoPass(vector<int>& heights) {
    int n = heights.size();

    // Compute NSE
    vector<int> nse(n, n);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && heights[st.top()] > heights[i]) {
            nse[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }

    // Clear stack for PSE
    while (!st.empty()) st.pop();

    // Compute PSE
    vector<int> pse(n, -1);
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && heights[st.top()] > heights[i]) {
            pse[st.top()] = i;
            st.pop();
        }
        st.push(i);
    }

    // Compute max area
    int maxArea = 0;
    for (int i = 0; i < n; i++) {
        int width = nse[i] - pse[i] - 1;
        maxArea = max(maxArea, heights[i] * width);
    }

    return maxArea;
}

int main() {

```

```

vector<int> heights = {2, 1, 5, 6, 2, 3};
cout << "Largest rectangle area: " << largestRectangleArea(heights) << "\n";
// 10
return 0;
}

```

8.11 Maximal Rectangle

```

#include <bits/stdc++.h>
using namespace std;

int largestRectangleArea(vector<int>& heights) {
    int n = heights.size();
    stack<int> st;
    int maxArea = 0;

    for (int i = 0; i <= n; i++) {
        int currHeight = (i == n) ? 0 : heights[i];
        while (!st.empty() && heights[st.top()] > currHeight) {
            int h = heights[st.top()];
            st.pop();
            int left = st.empty() ? -1 : st.top();
            maxArea = max(maxArea, h * (i - left - 1));
        }
        st.push(i);
    }
    return maxArea;
}

int maximalRectangle(vector<vector<char>>& matrix) {
    if (matrix.empty()) return 0;

    int rows = matrix.size(), cols = matrix[0].size();
    vector<int> heights(cols, 0);
    int maxArea = 0;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] == '1') {
                heights[j]++;
            } else {
                heights[j] = 0;
            }
        }
        maxArea = max(maxArea, largestRectangleArea(heights));
    }

    return maxArea;
}

int main() {

```

```

vector<vector<char>> matrix = {
    {'1', '0', '1', '0', '0'},
    {'1', '0', '1', '1', '1'},
    {'1', '1', '1', '1', '1'},
    {'1', '0', '0', '1', '0'}
};
cout << "Maximal rectangle area: " << maximalRectangle(matrix) << "\n"; // 6
return 0;
}

```

8.12 Stock Span

```

#include <bits/stdc++.h>
using namespace std;

vector<int> stockSpan(vector<int>& prices) {
    int n = prices.size();
    vector<int> span(n);
    stack<int> st; // stores indices

    for (int i = 0; i < n; i++) {
        while (!st.empty() && prices[st.top()] <= prices[i]) {
            st.pop();
        }
        span[i] = st.empty() ? (i + 1) : (i - st.top());
        st.push(i);
    }

    return span;
}

int main() {
    vector<int> prices = {100, 80, 60, 70, 60, 75, 85};
    vector<int> span = stockSpan(prices);

    for (int i = 0; i < prices.size(); i++) {
        cout << prices[i] << " -> " << span[i] << "\n";
    }
    return 0;
}

```

8.13 Asteroid Collision

```

#include <bits/stdc++.h>
using namespace std;

vector<int> asteroidCollision(vector<int>& asteroids) {
    stack<int> st;

```

```

    for (int asteroid : asteroids) {
        if (asteroid > 0) {
            // Moving right – push
            st.push(asteroid);
        } else {
            // Moving left – check for collisions
            while (!st.empty() && st.top() > 0 && st.top() < -asteroid) {
                st.pop(); // right-moving asteroid destroyed
            }

            if (st.empty() || st.top() < 0) {
                st.push(asteroid); // no collision or all right-movers gone
            } else if (st.top() == -asteroid) {
                st.pop(); // both destroyed
            }
            // else: st.top() > -asteroid, left-moving asteroid is destroyed (do
nothing)
        }
    }

    // Convert stack to vector
    vector<int> result(st.size());
    for (int i = st.size() - 1; i >= 0; i--) {
        result[i] = st.top();
        st.pop();
    }
    return result;
}

int main() {
    vector<int> asteroids1 = {5, 10, -5};
    vector<int> res1 = asteroidCollision(asteroids1);
    cout << "Asteroids: ";
    for (int x : res1) cout << x << " ";
    cout << "\n"; // 5 10

    vector<int> asteroids2 = {8, -8};
    vector<int> res2 = asteroidCollision(asteroids2);
    cout << "Asteroids: ";
    for (int x : res2) cout << x << " ";
    cout << "\n"; // (empty)

    vector<int> asteroids3 = {10, 2, -5};
    vector<int> res3 = asteroidCollision(asteroids3);
    cout << "Asteroids: ";
    for (int x : res3) cout << x << " ";
    cout << "\n"; // 10

    return 0;
}

```

8.14 Decode String

```
#include <bits/stdc++.h>
using namespace std;

string decodeString(string s) {
    stack<int> countStack;
    stack<string> stringStack;
    string currentString = "";
    int currentCount = 0;

    for (char c : s) {
        if (isdigit(c)) {
            currentCount = currentCount * 10 + (c - '0');
        } else if (c == '[') {
            countStack.push(currentCount);
            stringStack.push(currentString);
            currentString = "";
            currentCount = 0;
        } else if (c == ']') {
            int count = countStack.top(); countStack.pop();
            string prevString = stringStack.top(); stringStack.pop();

            string temp = "";
            for (int i = 0; i < count; i++) {
                temp += currentString;
            }
            currentString = prevString + temp;
        } else {
            currentString += c;
        }
    }

    return currentString;
}

int main() {
    vector<string> tests = {
        "3[a]2[bc]",
        "3[a2[c]]",
        "2[abc]3[cd]ef",
        "abc3[cd]xyz"
    };

    for (string s : tests) {
        cout << s << " -> " << decodeString(s) << "\n";
    }
    return 0;
}
```

8.15 Remove Duplicate Letters

```
#include <bits/stdc++.h>
using namespace std;

string removeDuplicateLetters(string s) {
    vector<int> freq(26, 0);
    vector<bool> visited(26, false);

    // Count frequency of each character
    for (char c : s) {
        freq[c - 'a']++;
    }

    stack<char> st;

    for (char c : s) {
        int idx = c - 'a';
        freq[idx]--; // Decrement remaining count

        if (visited[idx]) continue; // Already in stack, skip

        // While stack top is greater than current char
        // and stack top still appears later
        while (!st.empty() && st.top() > c && freq[st.top() - 'a'] > 0) {
            visited[st.top() - 'a'] = false;
            st.pop();
        }

        st.push(c);
        visited[idx] = true;
    }

    // Build result
    string result = "";
    while (!st.empty()) {
        result = st.top() + result;
        st.pop();
    }

    return result;
}

int main() {
    vector<string> tests = {
        "bcabc",
        "cbacdcbc",
        "abcd",
        "ecbacba"
    };

    for (string s : tests) {
        cout << s << " -> " << removeDuplicateLetters(s) << "\n";
    }
}
```

```
// bcabc -> abc
// cbacdcba -> acdb
// abcd -> abcd
// ecbacba -> each
return 0;
}
```

9. Python Implementation

9.1 Basic Stack Operations

```
# Python list as stack
stack = []

stack.append(10) # push
stack.append(20)
stack.append(30)

print("Top:", stack[-1]) # 30
print("Size:", len(stack)) # 3

stack.pop() # remove top
print("After pop, top:", stack[-1]) # 20

print("Is empty:", len(stack) == 0) # False
```

9.2 Balanced Parentheses

```
def is_valid(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '(': ')', '{': '}', '[': ']'}

    for c in s:
        if c in "({[":
            stack.append(c)
        else:
            if not stack or stack[-1] != pairs[c]:
                return False
            stack.pop()

    return len(stack) == 0

# Tests
tests = ["()", "()[]{}", "()", "([])", "{}[]"]
for s in tests:
    print(f"{s} -> {'Valid' if is_valid(s) else 'Invalid'}")
```

9.3 Next Greater Element

```
def next_greater_element(arr):
    n = len(arr)
    res = [-1] * n
    stack = [] # stores indices

    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            res[stack.pop()] = arr[i]
            stack.append(i)

    return res

arr = [4, 5, 2, 25]
print(next_greater_element(arr)) # [5, 25, 25, -1]
```

9.4 Previous Greater Element

```
def previous_greater_element(arr):
    n = len(arr)
    res = [-1] * n
    stack = [] # stores values

    for i in range(n):
        while stack and stack[-1] <= arr[i]:
            stack.pop()
        res[i] = stack[-1] if stack else -1
        stack.append(arr[i])

    return res
```

9.5 Next Smaller Element

```
def next_smaller_element(arr):
    n = len(arr)
    res = [-1] * n
    stack = [] # stores indices

    for i in range(n):
        while stack and arr[stack[-1]] > arr[i]:
            res[stack.pop()] = arr[i]
            stack.append(i)

    return res
```

9.6 Previous Smaller Element

```
def previous_smaller_element(arr):
    n = len(arr)
    res = [-1] * n
    stack = [] # stores indices

    for i in range(n):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()
        res[i] = arr[stack[-1]] if stack else -1
        stack.append(i)

    return res
```

9.7 Min Stack

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        if self.stack[-1] == self.min_stack[-1]:
            self.min_stack.pop()
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def get_min(self) -> int:
        return self.min_stack[-1]

ms = MinStack()
ms.push(3)
ms.push(5)
print(ms.get_min()) # 3
ms.push(2)
ms.push(1)
print(ms.get_min()) # 1
ms.pop()
print(ms.get_min()) # 2
```

9.8 Evaluate Postfix

```
def evaluate_postfix(tokens):
    stack = []
    operators = {'+', '-', '*', '/'}

    for token in tokens:
        if token in operators:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(int(a / b)) # integer division toward zero
        else:
            stack.append(int(token))

    return stack[-1]

print(evaluate_postfix(["2", "1", "+", "3", "*"])) # 9
```

9.9 Largest Rectangle in Histogram

```
def largest_rectangle_area(heights):
    n = len(heights)
    stack = []
    max_area = 0

    for i in range(n + 1):
        curr_height = 0 if i == n else heights[i]

        while stack and heights[stack[-1]] > curr_height:
            h = heights[stack.pop()]
            left = stack[-1] if stack else -1
            width = i - left - 1
            max_area = max(max_area, h * width)

        stack.append(i)

    return max_area

heights = [2, 1, 5, 6, 2, 3]
print(largest_rectangle_area(heights)) # 10
```

9.10 Maximal Rectangle

```

def maximal_rectangle(matrix):
    if not matrix:
        return 0

    rows, cols = len(matrix), len(matrix[0])
    heights = [0] * cols
    max_area = 0

    def largest_rectangle_area(heights):
        stack = []
        max_area = 0
        n = len(heights)

        for i in range(n + 1):
            curr = 0 if i == n else heights[i]
            while stack and heights[stack[-1]] > curr:
                h = heights[stack.pop()]
                left = stack[-1] if stack else -1
                max_area = max(max_area, h * (i - left - 1))
            stack.append(i)

        return max_area

    for i in range(rows):
        for j in range(cols):
            if matrix[i][j] == '1':
                heights[j] += 1
            else:
                heights[j] = 0
        max_area = max(max_area, largest_rectangle_area(heights))

    return max_area

matrix = [
    ['1', '0', '1', '0', '0'],
    ['1', '0', '1', '1', '1'],
    ['1', '1', '1', '1', '1'],
    ['1', '0', '0', '1', '0']
]
print(maximal_rectangle(matrix)) # 6

```

9.11 Stock Span

```

def stock_span(prices):
    n = len(prices)
    span = [0] * n
    stack = []

    for i in range(n):
        while stack and prices[stack[-1]] <= prices[i]:

```

```

        stack.pop()
        span[i] = i + 1 if not stack else i - stack[-1]
        stack.append(i)

    return span

prices = [100, 80, 60, 70, 60, 75, 85]
print(stock_span(prices)) # [1, 1, 1, 2, 1, 4, 6]

```

9.12 Asteroid Collision

```

def asteroid_collision(asteroids):
    stack = []

    for asteroid in asteroids:
        if asteroid > 0:
            stack.append(asteroid)
        else:
            while stack and stack[-1] > 0 and stack[-1] < -asteroid:
                stack.pop()

            if not stack or stack[-1] < 0:
                stack.append(asteroid)
            elif stack[-1] == -asteroid:
                stack.pop()

    return stack

print(asteroid_collision([5, 10, -5])) # [5, 10]
print(asteroid_collision([8, -8])) # []
print(asteroid_collision([10, 2, -5])) # [10]

```

9.13 Decode String

```

def decode_string(s: str) -> str:
    count_stack = []
    string_stack = []
    current_string = ""
    current_count = 0

    for c in s:
        if c.isdigit():
            current_count = current_count * 10 + int(c)
        elif c == '[':
            count_stack.append(current_count)
            string_stack.append(current_string)
            current_string = ""
            current_count = 0
        elif c == ']':

```

```

        count = count_stack.pop()
        prev_string = string_stack.pop()
        current_string = prev_string + current_string * count
    else:
        current_string += c

    return current_string

tests = ["3[a]2[bc]", "3[a2[c]]", "2[abc]3[cd]ef", "abc3[cd]xyz"]
for s in tests:
    print(f"{s} -> {decode_string(s)}")

```

9.14 Remove Duplicate Letters

```

def remove_duplicate_letters(s: str) -> str:
    from collections import Counter

    freq = Counter(s)
    visited = set()
    stack = []

    for c in s:
        freq[c] -= 1

        if c in visited:
            continue

        while stack and stack[-1] > c and freq[stack[-1]] > 0:
            visited.remove(stack.pop())

        stack.append(c)
        visited.add(c)

    return ''.join(stack)

tests = ["bcabc", "cbacdc", "abcd", "ecbacba"]
for s in tests:
    print(f"{s} -> {remove_duplicate_letters(s)}")

```

10. Code Explanation

10.1 Balanced Parentheses

How it works:

1. Iterate through each character.
2. If it's an opening bracket, push it onto the stack.
3. If it's a closing bracket:
 - Check stack is not empty (if empty → invalid)

- Pop the top and check if it matches the closing bracket using a mapping
- If mismatch → invalid

4. After processing all characters, stack must be empty (all brackets matched).

Edge cases handled:

- Empty string → returns true (stack is empty)
- Only closing brackets → stack empty check catches it
- Nested brackets → LIFO correctly matches innermost first
- Unmatched opening brackets → final empty check catches it

10.2 Next Greater Element (Left to Right)

How it works:

1. Maintain a **decreasing** stack of indices (values are in decreasing order from bottom to top).
2. For each element `arr[i]`:
 - While stack is not empty AND `arr[stack.top()] < arr[i]`:
 - The current element is the NGE for the element at the stack top
 - Pop and set result
 - Push current index
3. Elements still in stack at end have no NGE (remain -1).

Why decreasing stack? When we see a bigger element, it becomes the NGE for all smaller elements on the stack. The stack maintains the "waiting" elements whose NGE hasn't been found yet.

10.3 Largest Rectangle in Histogram (Single Pass)

How it works:

1. Maintain an **increasing** stack of indices (heights increase from bottom to top).
2. For each bar (including a sentinel 0 at the end):
 - While `heights[stack.top()] > currHeight`:
 - The bar at stack top is "bounded" — its NSE is the current index and its PSE is the new stack top
 - Pop it, compute area = height × (i - left - 1)
 - Update maxArea
 - Push current index
3. The sentinel 0 at the end forces all bars to be processed.

Why this works: When a shorter bar appears, it acts as the NSE for all taller bars on the stack. The PSE for each bar is the bar just below it on the stack (since indices on the stack are in increasing order of height).

10.4 Min Stack

How it works:

1. **Main stack** stores all elements.
2. **Min stack** stores the current minimum at each level.
3. `push(val)`: Push to main stack. If min stack is empty or `val <= minStack.top()`, push to min stack.

4. `pop()`: If top of both stacks match, pop from min stack too. Pop from main stack.
5. `getMin()`: Return min stack top.

Why this works: The min stack always has the current minimum at the top. When we push a value smaller than or equal to the current minimum, it becomes the new minimum. When we pop, if we're removing the current minimum, we also pop from the min stack to reveal the previous minimum.

10.5 Evaluate Postfix

How it works:

1. Scan tokens left to right.
2. If token is an operand (number), push it.
3. If token is an operator:
 - Pop two operands: `b` (first pop), `a` (second pop)
 - Apply operator: `a op b` (note: order matters for `-` and `/`)
 - Push result back
4. At end, stack top is the result.

Why order matters: In postfix `2 3 -`, the left operand is 2 and right operand is 3. So `2 - 3 = -1`. The first pop gives the right operand, the second pop gives the left operand.

10.6 Asteroid Collision

How it works:

1. Only push right-moving asteroids (positive) to stack.
2. For a left-moving asteroid (negative):
 - While stack top is right-moving AND smaller than the absolute value of current, pop (destroyed)
 - After collisions:
 - If stack empty or top is also left-moving → push current (no collision)
 - If stack top equals absolute value of current → pop both (both destroyed)
 - If stack top is larger → current is destroyed (do nothing)

Key insight: Only right-moving followed by left-moving causes collision. Right-after-right never collides. Left-after-left never collides. Left-after-right: the right was already processed and pushed, so when left comes, they collide.

10.7 Decode String

How it works:

1. Two stacks: `countStack` for repetition counts, `stringStack` for previous strings.
2. Iterate through each character:
 - **Digit:** Build the current count (handles multi-digit numbers like `12`).
 - `[` : Push current count and current string to stacks, reset both.
 - `]` : Pop count and previous string. Repeat current string `count` times, prepend previous string.
 - **Letter:** Append to current string.
3. Return current string.

Why two stacks? The nested structure means we need to save the "context" (previous string and count) when entering a new level of nesting. The stack naturally handles this LIFO pattern.

10.8 Remove Duplicate Letters

How it works:

1. Count frequency of each character in the string.
2. Maintain a **visited** set to track characters already in the stack.
3. For each character:
 - Decrement its frequency.
 - If already visited, skip.
 - While stack is not empty AND stack top > current AND stack top has remaining frequency > 0:
 - Pop stack top (we can place it later, so we prefer smaller character now)
 - Mark it as not visited
 - Push current character, mark visited.
4. Build result from stack.

Why this works: We want the lexicographically smallest string. When we see a smaller character, we can safely remove larger characters from the stack if they appear later. This is a form of monotonic stack (increasing order) combined with frequency tracking.

11. Complexity Analysis

General Stack Operations

Operation	Time Complexity	Space Complexity
<code>push()</code>	$O(1)$	$O(1)^*$
<code>pop()</code>	$O(1)$	$O(1)$
<code>top()</code> / <code>peek()</code>	$O(1)$	$O(1)$
<code>empty()</code>	$O(1)$	$O(1)$
<code>size()</code>	$O(1)$	$O(1)$

*Amortized $O(1)$ for dynamic array-based stack; may be $O(n)$ for a single push if reallocation happens.

Algorithm-Specific Complexity

Algorithm	Time Complexity	Space Complexity	Notes
Balanced Parentheses	$O(n)$	$O(n)$	Worst case: all opening brackets
Next Greater Element	$O(n)$	$O(n)$	Each element pushed/popped at most once
Previous Greater Element	$O(n)$	$O(n)$	Same as NGE

Algorithm	Time Complexity	Space Complexity	Notes
Next Smaller Element	$O(n)$	$O(n)$	Same as NGE
Previous Smaller Element	$O(n)$	$O(n)$	Same as NGE
Largest Rectangle in Histogram	$O(n)$	$O(n)$	Single pass or two-pass
Maximal Rectangle	$O(\text{rows} \times \text{cols})$	$O(\text{cols})$	Histogram per row
Min Stack	$O(1)$ per operation	$O(n)$	Two stacks approach
Evaluate Postfix	$O(n)$	$O(n)$	n = number of tokens
Evaluate Prefix	$O(n)$	$O(n)$	Same as postfix
Stock Span	$O(n)$	$O(n)$	Each element pushed/popped once
Asteroid Collision	$O(n)$	$O(n)$	Each asteroid pushed/popped at most once
Decode String	$O(n)$	$O(n)$	n = length of decoded string
Remove Duplicate Letters	$O(n)$	$O(1)$	Stack limited to 26 characters

Amortized Analysis

For monotonic stack algorithms, the time complexity is $O(n)$ **amortized**. While a single element may cause $O(n)$ pops, over the entire array, each element is pushed once and popped at most once, giving $O(n)$ total.

12. Common Patterns

Pattern 1: Next/Previous Greater/Smaller

How to identify: Problem asks for nearest greater/smaller element to left/right.

General approach: Use monotonic stack (decreasing for greater, increasing for smaller).

Template:

```
Initialize stack, result array with -1
For each element:
    While stack not empty AND condition:
        Pop and resolve
    Push current
```

Examples: NGE, PGE, NSE, PSE, Stock Span

Pattern 2: Largest Rectangle in Histogram

How to identify: Problem involves finding largest rectangular area in a 1D or 2D array.

General approach: For each bar, find NSE and PSE. Area = height $\times$ (NSE - PSE - 1).

Variations:

- Maximal Rectangle (2D $\rightarrow$ per row histogram)
- Trapping Rain Water (related but uses left/right max, not stack)

Pattern 3: Expression Evaluation

How to identify: Problem involves evaluating mathematical expressions.

General approach:

- Postfix: One stack, scan left to right
- Infix: Two stacks (operand and operator), or convert to postfix first
- Prefix: One stack, scan right to left

Pattern 4: Parsing Nested Structures

How to identify: Problem involves nested brackets, tags, or encoded strings.

General approach: Use stack to track nesting depth. May need multiple stacks for different types of data.

Examples: Decode String, Remove Duplicate Letters, HTML parsing

Pattern 5: Simulation with LIFO Order

How to identify: Problem involves sequential operations where order matters and reversal is needed.

General approach: Simulate the process step by step. Use stack to keep track of "active" elements.

Examples: Asteroid Collision, Validate Stack Sequences, Car Fleet

Pattern 6: Stack + Frequency/HashMap

How to identify: Problem involves both ordering constraints and frequency/distance constraints.

General approach: Combine stack with frequency array or hashmap for additional constraints.

Examples: Remove Duplicate Letters, Remove K Digits

Pattern 7: Two Stacks / Auxiliary Stack

How to identify: Problem requires additional operations beyond push/pop (getMin, getMax, queue behavior).

General approach: Use a second stack to maintain auxiliary information.

Examples: Min Stack, Queue using Two Stacks, Max Stack

13. Common Mistakes

13.1 Balanced Parentheses

- **Forgetting to check stack is empty before pop** → Segmentation fault or wrong answer
- **Not checking stack is empty after loop** → Unmatched opening brackets
- **Using == instead of matching pairs** → Must map opening to closing correctly
- **Only checking length parity** → `([])` has even length but is invalid

13.2 Next Greater Element

- **Using value stack instead of index stack** → Can't compute NGE for all elements
- **Incorrect comparison direction** → `arr[top] < arr[i]` vs `arr[top] > arr[i]`
- **Forgetting to initialize result with -1** → Elements without NGE need -1
- **Non-strict vs strict comparison** → NGE usually strict (`>`, not `>=`), but depends on problem

13.3 Largest Rectangle in Histogram

- **Not using sentinel** → Last few bars may not be processed
- **Incorrect width calculation** → `width = NSE - PSE - 1`, not `NSE - PSE`
- **Forgetting PSE** → Using only NSE or NSE alone gives wrong width
- **Integer overflow** → Height × width may exceed int range (use `long long`)

13.4 Min Stack

- **Not pushing duplicate min values** → If min value appears twice, popping removes it from min stack but the other is still there
- **Using `<=` vs `<` in push condition** → Must use `<=` to handle duplicates correctly
- **Forgetting to sync pop** → Min stack must be popped only when its top matches main stack top

13.5 Evaluate Postfix

- **Wrong operand order** → `a = pop(), b = pop(),` then `a op b` (not `b op a`)
- **Integer division toward zero** → C++ truncates toward zero, Python `//` floors (need `int(a/b)`)
- **Not handling multi-digit numbers** → Need to parse multiple digits as one number

13.6 Asteroid Collision

- **Forgetting to check after popping** → After pop, the next stack top may also collide
- **Only checking one collision** → Must loop until no more collisions
- **Incorrect size comparison** → Compare absolute values, not signed values
- **Not handling equal-sized collision** → Both destroyed

13.7 Decode String

- **Not handling multi-digit counts** → Need to accumulate: `currentCount = currentCount * 10 + digit`
- **Incorrect string concatenation order** → `prevString + repeatedString`, not `repeatedString + prevString`
- **Not resetting after [** → Must reset `currentString` and `currentCount` after pushing

13.8 Remove Duplicate Letters

- **Forgetting to decrement frequency** → Frequency tracks remaining occurrences
- **Using visited check incorrectly** → Already visited characters should be skipped, not processed again
- **Incorrect pop condition** → Only pop if stack top has remaining frequency > 0
- **Forgetting to update visited on pop** → Must set visited to false when removing from stack

13.9 General Mistakes

- **Using array as stack without tracking size** → Need a separate **top** variable
- **Stack overflow with recursion** → Use explicit stack for deep recursion
- **Amortized complexity misunderstanding** → $O(n)$ amortized is good, but worst-case per operation can be $O(n)$
- **Modifying stack while iterating** → Don't modify the container being iterated over
- **Using **stack** from STL but needing iteration** → STL stack doesn't support iteration; use **deque** or **vector** if needed

14. Edge Cases

Input-Based Edge Cases

Edge Case	Where It Matters	Expected Behavior
Empty input	All algorithms	Return empty/0/true appropriately
Single element	NGE/PGE/NSE/PSE, histogram	Return -1 or height itself
All equal elements	Monotonic stack	Depends on strict/non-strict comparison
Sorted ascending	NGE: all have NGE, PGE: only first has -1	Works correctly
Sorted descending	NGE: only last has -1, PGE: all have PGE	Works correctly
All same value	Histogram: area = value × n	Must handle non-strict in PSE computation
Large values (10^9)	Histogram, expression evaluation	Use long long to avoid overflow
Negative values	Expression evaluation, asteroid	Handle sign correctly
Duplicate values	Remove duplicate letters, monotonic stack	Depends on strict vs non-strict
Very long encoded string	Decode string	Result string may be large
Single character	Remove duplicate letters	Return that character
Already valid string	Balanced parentheses	Return true
Only one type of bracket	Balanced parentheses	Still works

Algorithm-Specific Edge Cases

Balanced Parentheses

- "" → true (empty string)
- "(" → false (unmatched)
- ")" → false (closing without opening)
- "([])" → false (wrong nesting)
- "{[]}" → true (correct nesting)

Largest Rectangle in Histogram

- [0] → 0
- [1] → 1
- [1, 2, 3, 4, 5] → 9 (3×3 middle, or 5×1=5, etc. Actually max is (1+2+3+4+5) = 15... wait, 5 bars of height 1 = 5, or 4 bars... let me think: heights [1,2,3,4,5], max area = 9 (3×3 from indexes 2-4 with height 3))
- [5, 4, 3, 2, 1] → 9 (same, symmetric)
- [1000000000, 1000000000, 1000000000] → 3000000000 (use long long)

Asteroid Collision

- [5, -5] → [] (both destroyed)
- [5, 10, -15] → [-15] (big left-moving destroys all)
- [-5, -10, 5] → [-5, -10, 5] (no collisions, all moving left then right)
- [1, -2, -2, -2] → [-2, -2, -2] (first right-moving destroyed)

Decode String

- "" → ""
- "3[a]" → "aaa"
- "3[a2[b]]" → "abbabbabb"
- "10[a]" → "aaaaaaaaaa" (multi-digit count)
- "abc" → "abc" (no encoding)

Remove Duplicate Letters

- "a" → "a"
- "aa" → "a"
- "abab" → "ab"
- "bbcaac" → "bac"
- "abacb" → "abc"

15. Variations

15.1 Monotonic Queue (Deque)

What changes: Uses deque instead of stack, maintains monotonic order in a sliding window.

When used: Sliding window maximum/minimum, problems where the window size is fixed.

Importance: High for placements. Very common in hard problems.

15.2 Stack with getMin() O(1) Space

What changes: Instead of an auxiliary stack, encode the minimum in the stored value. Store $2*val - min$ when updating min.

When used: When space optimization is required.

Limitation: May overflow for large values.

15.3 Queue Using Two Stacks

What changes: Two stacks simulate FIFO behavior. Push to one stack, pop from the other.

When used: When you need queue operations but only have stack API.

Complexity: O(1) amortized per operation.

15.4 Stack with Middle Element Access

What changes: Use a doubly-linked list or two stacks (like a deque) to access the middle element.

When used: Specialized problems.

15.5 Multiple Stacks in One Array

What changes: Partition a single array into multiple stacks with dynamic boundaries.

When used: Memory-constrained environments.

15.6 Stack with Max Operation

What changes: Same as Min Stack but tracks maximum instead.

When used: Problems requiring max tracking.

15.7 Infix to Postfix Conversion

What changes: Uses operator stack to convert infix to postfix. Handles operator precedence and associativity.

When used: Expression evaluation, calculator implementations.

Importance: High for placements.

15.8 Trapping Rain Water (Stack Version)

What changes: Uses stack to find boundaries that trap water. Similar to largest rectangle but with different area calculation.

When used: Water trapping problems. Note: Two-pointer approach is often more efficient.

16. Related Algorithms/Data Structures

Stack vs Queue

Aspect	Stack	Queue
Order	LIFO	FIFO
Primary use	Reversal, parsing, backtracking	Order preservation, BFS, scheduling
Key algorithms	Monotonic stack, DFS	Monotonic queue, BFS
Sliding window	Not suitable	Suitable (deque)

Stack vs Deque

- **Deque** can function as both stack and queue
- C++ STL **stack** is a container adapter over **deque** by default
- Use **deque** directly when you need both push/pop on both ends
- Use **stack** when you want only LIFO semantics (cleaner code)

Stack vs Heap (Priority Queue)

Aspect	Stack	Heap
Order	Insertion order (LIFO)	Sorted order by priority
Top element	Most recently added	Highest/lowest priority
Use case	Nested structures, reversal	Dynamic ordering, scheduling
Monotonic	Yes (monotonic stack)	No

Stack vs Recursion

- **Recursion** uses the call stack implicitly
- **Explicit stack** can simulate recursion iteratively (avoid stack overflow)
- Use explicit stack when:
 - Recursion depth may exceed stack limit
 - Need to pause/resume computation
 - Need to traverse without function call overhead

Stack vs Segment Tree / Fenwick Tree

- **Stack** is for local relationships (next/previous element)
- **Segment Tree** is for range queries (min, max, sum over any range)
- If the problem is "find next greater element in range" → monotonic stack
- If the problem is "find the maximum in any subarray" → segment tree

NGE/PGE/NSE/PSE Patterns vs Two Pointers

- **Two pointers** can solve some of these problems but at $O(n^2)$ for general case
- **Monotonic stack** is the standard $O(n)$ solution
- Example: Trapping Rain Water can be solved with two pointers ($O(n)$, $O(1)$) or stack ($O(n)$, $O(n)$)

17. Practice Problems

Easy

Problem	Platform	Pattern	Notes
Valid Parentheses	LeetCode 20	Balanced parentheses	Classic starter. Must know.
Min Stack	LeetCode 155	Auxiliary stack	O(1) getMin design. Very common.

Medium

Problem	Platform	Pattern	Difficulty
Next Greater Element I	LeetCode 496	Monotonic stack	Easy-Medium
Asteroid Collision	LeetCode 735	Stack simulation	Medium
Decode String	LeetCode 394	Two stacks	Medium
Stock Span	GFG	Monotonic stack	Medium
Remove Duplicate Letters	LeetCode 316	Stack + frequency	Medium
Evaluate Reverse Polish Notation	LeetCode 150	Postfix evaluation	Medium

Hard

Problem	Platform	Pattern	Difficulty
Largest Rectangle in Histogram	LeetCode 84	Monotonic stack	Hard
Maximal Rectangle	LeetCode 85	2D → 1D histogram	Hard
Remove K Digits	LeetCode 402	Monotonic stack variant	Medium
Trapping Rain Water	LeetCode 42	Stack / Two pointers	Hard
Car Fleet	LeetCode 853	Monotonic stack	Medium
Sum of Subarray Minimums	LeetCode 907	NSE + PSE contributions	Medium-Hard
Longest Valid Parentheses	LeetCode 32	Stack or DP	Hard

18. Interview Explanation

General Stack

"A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. The primary operations are push (add to top), pop (remove from top), and top (peek at top), all in O(1) time. Stacks are used wherever we need to process things in reverse order or handle nested structures — like function calls, undo operations, and parsing brackets."

Balanced Parentheses

"For balanced parentheses, I maintain a stack. When I see an opening bracket, I push it. When I see a closing bracket, I check if the stack top matches it. If it does, I pop; otherwise, it's invalid. At the end, the stack must be empty. This is $O(n)$ time and $O(n)$ space. The key insight is that LIFO naturally handles the nesting — the most recent opening bracket must match the next closing bracket."

Monotonic Stack

"A monotonic stack maintains elements in sorted order — either increasing or decreasing. When I process a new element, I pop elements from the stack that violate the monotonic property. This is powerful because it reduces many $O(n^2)$ problems to $O(n)$ — each element is pushed once and popped at most once. The classic example is Next Greater Element: maintain a decreasing stack, and when a larger element appears, it becomes the next greater for all smaller elements on the stack."

Largest Rectangle in Histogram

"For the largest rectangle in histogram, I use a monotonic increasing stack. For each bar, I want to find the next smaller element to the right and the previous smaller element to the left. The width of the rectangle that uses this bar is NSE index minus PSE index minus 1. The area is height times width. I compute this for all bars in $O(n)$ using a single pass with a stack. The sentinel 0 at the end ensures all bars are processed."

Min Stack

"For Min Stack, I use two stacks — the main stack stores all elements, and an auxiliary stack stores the current minimum at each level. On push, I compare the new value with the current minimum and push to the min stack if it's smaller or equal. On pop, I sync both stacks. Both operations stay $O(1)$."

19. Revision Notes

Core Stack

- **LIFO**: Last In, First Out
- **Operations**: push, pop, top, empty — all $O(1)$
- **STL**: `stack<T>` (adapter over `deque` by default)
- **Python**: Use list — `append()` for push, `pop()` for pop, `[-1]` for top

Balanced Parentheses

- Push opening brackets, match closing brackets with stack top
- Must check: stack empty before pop, stack empty after loop
- Use map for bracket pairs

Monotonic Stack — Quick Reference

Pattern	Stack Order	Traversal	Comparison
NGE	Decreasing	Left to right	<code>arr[top] < arr[i] → pop</code>

Pattern	Stack Order	Traversal	Comparison
PGE	Decreasing	Left to right	<code>arr[top] <= arr[i] → pop</code>
NSE	Increasing	Left to right	<code>arr[top] > arr[i] → pop</code>
PSE	Increasing	Left to right	<code>arr[top] >= arr[i] → pop</code>

Memory tip: NGE = Next Greater = Decreasing stack (smaller elements on top, waiting for a bigger element to resolve them). **NSE** = Next Smaller = Increasing stack (bigger elements on top, waiting for a smaller element to resolve them).

Largest Rectangle in Histogram

- **Single pass:** Add sentinel 0, maintain increasing stack
- **Area formula:** `height * (i - stack.top() - 1)` after popping
- **Key:** When a smaller bar appears, it bounds the height of taller bars
- **Use long long** for area

Maximal Rectangle

- Convert each row to histogram heights
- Heights: increment if cell is '1', reset to 0 if '0'
- Run histogram algorithm on each row
- $O(\text{rows} \times \text{cols})$ time, $O(\text{cols})$ space

Min Stack

- Two stacks: main + min
- Push: `if val <= minStack.top(): minStack.push(val)`
- Pop: `if mainStack.top() == minStack.top(): minStack.pop()`

Postfix Evaluation

- Scan left to right
- Operand → push; Operator → pop b, pop a, apply a op b, push result
- Order matters for - and /: a is first popped operand's complement

Decode String

- Two stacks: `countStack`, `stringStack`
- `[` → save context; `]` → restore and repeat
- Multi-digit count: `count = count * 10 + digit`

Remove Duplicate Letters

- Stack + frequency + visited
- Pop larger chars if they appear later
- Result is lexicographically smallest

20. Final Cheat Sheet

Stack at a Glance

STACK CHEAT SHEET

PRINCIPLE: LIFO (Last In, First Out)

OPERATIONS:

```
push(x)  → O(1)   Insert at top
pop()    → O(1)   Remove from top
top()    → O(1)   Peek at top
empty()  → O(1)   Check if empty
```

WHEN TO USE:

- Nested structures (parentheses, tags)
- Reversal problems
- "Next/Previous Greater/Smaller Element" problems
- Expression evaluation (postfix, prefix)
- Simulation with LIFO order (asteroid collision)
- Backtracking, DFS, undo operations

MONOTONIC STACK:

- Decreasing stack → NGE, PGE (pop when $\text{arr}[\text{top}] < \text{arr}[i]$)
- Increasing stack → NSE, PSE (pop when $\text{arr}[\text{top}] > \text{arr}[i]$)
- Each element pushed/popped once → $O(n)$ amortized
- Key: The element being popped finds its answer in $\text{arr}[i]$

LARGEST RECTANGLE IN HISTOGRAM:

```
for i = 0 to n (with sentinel 0 at n):
    while stack not empty AND heights[top] > heights[i]:
        h = heights[pop()]
        left = stack.empty() ? -1 : stack.top()
        area = max(area, h * (i - left - 1))
    push(i)
return area
```

MIN STACK:

```
push(x): main.push(x); if x <= min.top(): min.push(x)
pop():   if main.top() == min.top(): min.pop(); main.pop()
getMin(): return min.top()
```

POSTFIX:

```
for token in tokens:
    if token is operator: b = pop(); a = pop(); push(a op b)
    else: push(int(token))
```

DECODE STRING:

```
for c in s:
    if isdigit: count = count*10 + (c-'0')
```

```
elif '[': push count; push currStr; reset
elif ']': pop -> repeat currStr; prepend popped string
else: currStr += c
```

REMOVE DUPLICATE LETTERS:

```
freq = count(s)
for c in s:
    freq[c]--
    if visited[c]: continue
    while stack not empty AND stack.top > c AND freq[top] > 0:
        visited[stack.pop()] = false
    push(c); visited[c] = true
return ''.join(stack)
```

EDGE CASES TO TEST:

- Empty input
- Single element
- All equal values
- Sorted ascending / descending
- Large values (overflow)
- Duplicates (strict vs non-strict comparison)

COMPLEXITY SUMMARY:

Algorithm	Time	Space
Balanced Parentheses	$O(n)$	$O(n)$
NGE / PGE / NSE / PSE	$O(n)$	$O(n)$
Histogram	$O(n)$	$O(n)$
Maximal Rectangle	$O(R \cdot C)$	$O(C)$
Min Stack	$O(1)^*$	$O(n)$
Postfix Evaluation	$O(n)$	$O(n)$
Decode String	$O(n)$	$O(n)$
Remove Duplicate Letters	$O(n)$	$O(1)$
Asteroid Collision	$O(n)$	$O(n)$

*Amortized $O(1)$ per operation for monotonic stack algorithms

This guide covers all major stack algorithms for placements and competitive programming. Master the monotonic stack — it's the single most important pattern that separates good from great solutions in coding interviews.